

T1.3.E — Bridge landing → backend con HMAC

Implementación · Rama `pr/t1.3.e-bridge-landing-backend` · Commit `45da6a1` · 2026-05-02

Status: Branch pusheado a GitHub. Build OK. **NO mergeado a main.** Owner aprueba via PR en GitHub UI.

Antes de mergear: verificar que **BACKEND_URL** e **INTERNAL_BRIDGE_SECRET** estén configuradas en Vercel. Sin **BACKEND_URL** el bridge falla en cada webhook → Stripe reintenta infinitamente y vas a ver alertas.

1. Archivos cambiados

Archivo	Δ líneas	Tipo	Rol
landing/lib/internal-bridge.ts	+117 / -0	A	helper HMAC + fetch al backend
landing/app/api/webhook/route.ts	+18 / -0	M	import + llamada al bridge tras el switch
TOTAL T1.3.E	+135 / -0	2 archivos	

2. Resumen del diff

a) Helper landing/lib/internal-bridge.ts (nuevo)

Función pública:

```
export type BridgeResult = {
  ok: boolean;
  status?: number; // status del backend o undefined si fetch falló
  error?: string; // mensaje opaco (sin secrets)
};

export async function reenviarEventoStripeABackend(
  evento: unknown,
): Promise
```

Lógica del helper:

- Lee **BACKEND_URL** e **INTERNAL_BRIDGE_SECRET** de env. Si falta cualquiera, loguea y retorna **ok=false** (no intenta el fetch).
- Hace **JSON.stringify(evento)** **una sola vez**. El mismo string se usa para HMAC y para el body del fetch — garantiza que firma y body coincidan exactamente.
- Computa **HMAC-SHA256(body, secret).digest('hex')** y lo manda en header *X-Internal-Signature*.
- Timeout 8s vía **AbortController** (Stripe expira a 10s).
- **ok=true** solo si backend responde 2xx. 401/400/500/timeout/fetch fail → **ok=false**.
- Logs nunca incluyen el secret ni el body completo (max 500 chars de preview del body de error del backend).

b) Integración en route.ts

```
// Después del switch existente (welcome WhatsApp se mantiene):
const bridge = await reenviarEventoStripeABackend(event);
```

```
if (!bridge.ok) {
  console.error(
    `[WEBHOOK] Bridge a backend falló (status=${bridge.status ?? "n/a"} `
    + `error=${bridge.error}). Respondiendo 500 a Stripe para retry.`
  );
  return NextResponse.json(
    { error: "bridge_failed", reason: bridge.error },
    { status: 500 },
  );
}
return NextResponse.json({ received: true });
```

3. Reglas de respuesta a Stripe (la decisión clave)

Backend respondió	Bridge ok?	Landing → Stripe	Razón
2xx (200, 201, etc)	true	200	Procesado OK
401	false	500	Secret desincronizado — alerta + retry
400	false	500	Bug de bridge — alerta + retry
500 retryable (race)	false	500	Backend pidió retry — propagar
Timeout (>8s)	false	500	Red lenta o backend caído
DNS/network fail	false	500	Backend no alcanzable
BACKEND_URL faltante	false	500	Misconfig — alerta + retry
SECRET faltante	false	500	Misconfig — alerta + retry

Política: en duda, devolver 500 a Stripe. Stripe reintenta el evento varias veces con backoff exponencial; el peor caso es que veamos eventos repetidos en el dashboard de Stripe. La idempotencia del backend (T1.3.C: event.id + invoice.id) protege contra acreditar doble. Lo opuesto — devolver 200 con bridge fallido — perdería eventos silenciosamente y el usuario no recibiría sus créditos.

4. Flujo end-to-end (post T1.3.E)



5. Comandos ejecutados y resultado

Comando	Resultado
npm install	OK · 146 packages instalados · 2 vulnerabilidades preexistentes
npx tsc --noEmit	0 errores en internal-bridge.ts y route.ts (que mi cambio agrega)
npm run lint	0 findings en internal-bridge.ts · 16 warnings/errors preexistentes (any en otros archivos, img tag, etc.) — no
npm run build	✓ Compiled successfully · ✓ TypeScript pasa · ✓ Static pages OK · /api/webhook registrado como dynamic

Output relevante de npm run build

```
✓ Compiled successfully in 1829ms
Running TypeScript ...
Finished TypeScript in 1917ms ...
✓ Generating static pages using 14 workers (12/12) in 440ms

Route (app)
■ f /api/webhook           ← endpoint que actualizamos
■ f /api/checkout
■ f /api/whatsapp-webhook
...
```

6. Riesgos y blockers

Sin blockers para mergear

Build pasa. Mi código nuevo está libre de errores tsc/lint. Los errores preexistentes en otros archivos (any types, img tag) ya estaban en main antes de T1.3.E.

Riesgos operacionales (post-merge)

- **Welcome WhatsApp duplicado en retry:** el welcome se envía antes del bridge. Si el bridge falla, Stripe reintenta y el welcome se vuelve a enviar. Mitigación: el usuario recibe a lo sumo 2-3 welcomes en una ventana de minutos. Aceptable; no rompe nada.
- **Secret desincronizado:** si Vercel y Render tienen valores distintos de INTERNAL_BRIDGE_SECRET, todos los webhooks devuelven 401 → 500 a Stripe → retries infinitos. Detección: Stripe Dashboard mostrará webhooks con state=Failing.
- **BACKEND_URL apuntando a sitio caído:** timeout en cada webhook (8s × 3 retries = ~24s por evento). Stripe deshabilita el endpoint tras suficientes fallos.
- **Eventos legacy** (paquetes one-time): el bridge los reenvía al backend igual; procesar_evento_suscripcion los responde con 'tipo no soportado: ...' (200 OK no-retryable). Sin efecto.

7. Próximos pasos para el owner

a) Verificar env vars en Vercel

- Settings → Environment Variables, scope **Production**:
- **BACKEND_URL** = https://dona-agent.onrender.com
- **INTERNAL_BRIDGE_SECRET** = (mismo valor que en Render)

b) Mergear T1.3.E

- PR T1.3.E: <https://github.com/celestinojbm/Dona-agent/compare/main...pr/t1.3.e-bridge-landing-backend>
- Vercel redespiega el landing automáticamente al merge.
- Stripe Dashboard sigue apuntando a la landing — no se toca.

c) Verificar end-to-end

- Hacer una compra de prueba (modo test si Stripe está en test, o real con saldo bajo de test en producción).
- Confirmar en Render logs: *[BILLING] Acreditado X cr* → ...
- Confirmar en Stripe Dashboard → Webhooks → tu endpoint: status 200.
- Confirmar saldo del telefono via *dona saldo* por WhatsApp.

d) T1.3.F (opcional)

Mover Stripe Dashboard URL al backend directo (<https://dona-agent.onrender.com/webhook/stripe-suscripcion> o similar) y eliminar el bridge. Reduce un hop pero requiere cambio en Stripe Dashboard. **Solo** hacerlo cuando T1.3.E esté estable y soakeado varias semanas con tráfico real.